

TEMA N° 4



TEMA 4 APLICACIÓN PRÁCTICA Y CONEXIÓN IOT (MICROCONTROLADORES)



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Comprender e integrar** el ecosistema de hardware libre mediante el uso de microcontroladores y su entorno de desarrollo, aplicándolo a la resolución de problemas técnicos del entorno productivo y civil.
2. **Diseñar e implementar** sistemas de comunicación inalámbrica estables entre dispositivos de hardware y aplicaciones móviles, garantizando el flujo bidireccional de datos (comandos y lecturas de sensores).
3. **Desarrollar software móvil** estructurado para el control perimetral y monitoreo en tiempo real, consolidando tu perfil como Técnico en Sistemas Informáticos capaz de transformar necesidades del entorno en soluciones tecnológicas tangibles.



PRÁCTICA



PREGUNTA DETONANTE

¿Cómo podemos lograr que un teléfono inteligente, con toda su capacidad de procesamiento, interactúe físicamente con las herramientas, infraestructuras y sistemas productivos de nuestra propia comunidad sin depender completamente de una costosa conexión fija a Internet?

EL DESAFÍO EN EL BARRIO SAÓ Y LOS DISTRITOS DE PAILÓN

Imagina que has sido seleccionado para diseñar el sistema de automatización y monitoreo del nuevo módulo educativo del CEA Herman Ortiz Camargo, ubicado en el Barrio Saó de la población de Pailón. Este nuevo edificio cuenta con laboratorios de computación equipados, pero enfrenta dos problemas críticos típicos de la zona: las elevadas temperaturas térmicas que ponen en riesgo los servidores y equipos informáticos, y la necesidad de optimizar el consumo eléctrico del sistema de iluminación exterior e interior, previniendo descuidos que dejen luces encendidas durante los fines de semana.

Simultáneamente, comerciantes del Barrio Central Fátima (zona de alta actividad comercial formal frente a la iglesia) y productores agrícolas que despachan cargamentos desde la estación de trenes en el Barrio 24 de Septiembre han manifestado su interés en este proyecto. Los comerciantes necesitan controlar las persianas de seguridad y las alarmas de sus negocios de forma remota desde sus teléfonos móviles sin usar infraestructuras complejas, mientras que en la zona ferroviaria se requiere monitorear de manera constante la temperatura de los contenedores de almacenamiento antes de su distribución.

Como Técnico en Sistemas Informáticos, te encuentras ante la tarea de crear un prototipo funcional que use un microcontrolador central conectado a sensores de temperatura y actuadores de potencia (relés para luces y motores). El sistema debe comunicarse mediante señales inalámbricas directas con una aplicación móvil instalada en el dispositivo del personal de seguridad o del administrador del centro. El problema radica en cómo estructurar el código del microcontrolador y la lógica de la aplicación móvil para que la transmisión de datos sea limpia,



inmediata, segura y no sufra desconexiones debido a las interferencias o las distancias dentro de los predios del CEA o los almacenes de Pailón.



TEORÍA



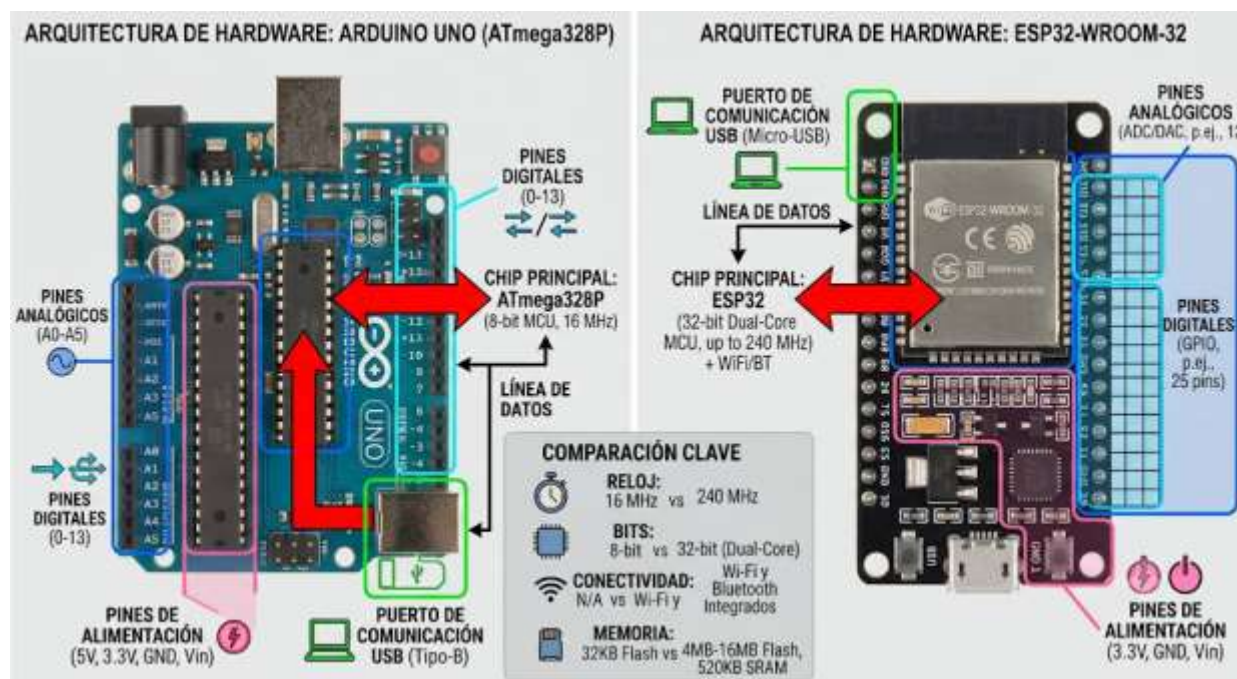
4.1. INTRODUCCIÓN AL ECOSISTEMA ARDUINO Y SU IDE

El desarrollo de soluciones en el Internet de las Cosas (IoT) requiere de una base de hardware flexible, económica y de código abierto. El ecosistema Arduino representa este estándar industrial y educativo. Un microcontrolador es un circuito integrado programable que contiene todos los componentes de una computadora de forma miniatura: unidad de procesamiento (CPU), memoria RAM, memoria Flash (donde se almacena el programa) y pines de entrada y salida.

A diferencia de una computadora convencional que ejecuta un sistema operativo complejo, un microcontrolador ejecuta un único bucle de programa de manera dedicada y repetitiva, lo que garantiza tiempos de respuesta exactos y un consumo energético mínimo. Para escribir y transferir código a estos dispositivos, utilizamos el Entorno de Desarrollo Integrado (IDE) de Arduino. Este software proporciona un editor de texto con resaltado de sintaxis, un compilador que traduce el código escrito en un lenguaje basado en C/C++ a código de máquina, y un monitor serie para depurar datos en tiempo real.

El ciclo de ejecución de cualquier programa en este ecosistema se divide en dos funciones obligatorias:

1. `setup()`: Se ejecuta una sola vez cuando el dispositivo se enciende o se reinicia. Se utiliza para configurar el comportamiento inicial de los pines, inicializar comunicaciones serie o activar módulos específicos.
2. `loop()`: Es un bucle infinito que se ejecuta inmediatamente después de que termina `setup()`. Contiene la lógica principal del sistema, leyendo sensores y activando respuestas continuamente.



4.2. PROGRAMACIÓN DE ENTRADAS Y SALIDAS DIGITALES EN MICROCONTROLADORES

Los pines de un microcontrolador actúan como los ojos y las manos del sistema informático. Para interactuar con el entorno del CEA Herman Ortiz Camargo, debemos clasificar estos pines en dos categorías operativas:

1. **Entradas Digitales:** Permiten al microcontrolador leer un estado del mundo exterior. Al ser "digital", solo reconoce dos niveles de voltaje: ALTO (HIGH, típicamente 5V o 3.3V) y BAJO (LOW, 0V). Un ejemplo práctico es un interruptor magnético instalado en la puerta del laboratorio informático en el Barrio Saó para detectar si la puerta está abierta o cerrada.
2. **Salidas Digitales:** Permiten al microcontrolador enviar una señal eléctrica para modificar el entorno. Al poner un pin en HIGH, suministramos energía; al ponerlo en LOW, la cortamos. Dado que los pines del microcontrolador manejan corrientes muy bajas (máximo 20 a 40 mA), para activar cargas grandes como reflectores en el coliseo municipal del Barrio 27 de Mayo o extractores de aire, se emplean módulos de relé que actúan como interruptores electrónicos aislados.



Para garantizar lecturas de entrada estables, es obligatorio comprender el concepto de resistencias de *Pull-Up* o *Pull-Down*. Si un pin de entrada se deja desconectado en el aire, actuará como una antena captando ruido electromagnético, provocando falsas lecturas entre HIGH y LOW. Una configuración de resistencia asegura que, cuando el sensor no esté activo, el pin esté firmemente conectado a un voltaje de referencia conocido.

A continuación, se presenta el código fuente en lenguaje C++ para el microcontrolador que gestiona la alerta de temperatura y el control de iluminación exterior del módulo educativo en Pailón:

C++

```
// Definición de constantes para asignar los números de pines físicos
const int pinSensorPuerta = 2;
// Pin de entrada para el interruptor del laboratorio
const int pinExtractorAire = 13;
// Pin de salida para activar el relé del extractor
// Variable para almacenar el estado de la puerta
int estadoPuerta = 0;
void setup() {
    // Configura el pin del sensor como entrada utilizando la resistencia Pull-Up
    // interna del chip // Esto asegura que el pin lea HIGH por defecto cuando el
    // interruptor esté abierto pinMode(pinSensorPuerta, INPUT_PULLUP);
    // Configura el pin del extractor como una salida digital para enviar voltaje al
    // relé pinMode(pinExtractorAire, OUTPUT);
    // Inicializa la comunicación de datos en serie a una velocidad estándar de 9600
    // bits por segundo Serial.begin(9600);
}
void loop() {
    // Lee el estado eléctrico actual del pin conectado al sensor de la puerta
    estadoPuerta = digitalRead(pinSensorPuerta);
    // Condicional para evaluar si la puerta ha sido vulnerada o abierta // Debido
    // a INPUT_PULLUP, un estado LOW significa que el interruptor cerró el circuito a
    // tierra (puerta cerrada) if (estadoPuerta == LOW) {
    // Si el circuito está cerrado, la estructura está protegida. Apagamos el
    // extractor o alarma. digitalWrite(pinExtractorAire, LOW);
    Serial.println("
    Sistema Seguro: Puerta del laboratorio cerrada.");
    }
    else {
        // Si el circuito se abre (estado HIGH), significa que la puerta se abrió en el
        // Barrio Saó // Activamos inmediatamente el pin de salida para encender el
        // extractor o la alerta visual digitalWrite(pinExtractorAire, HIGH);
        Serial.println("
        ALERTA: Puerta del laboratorio detectada ABIERTA.");
    }
}
```



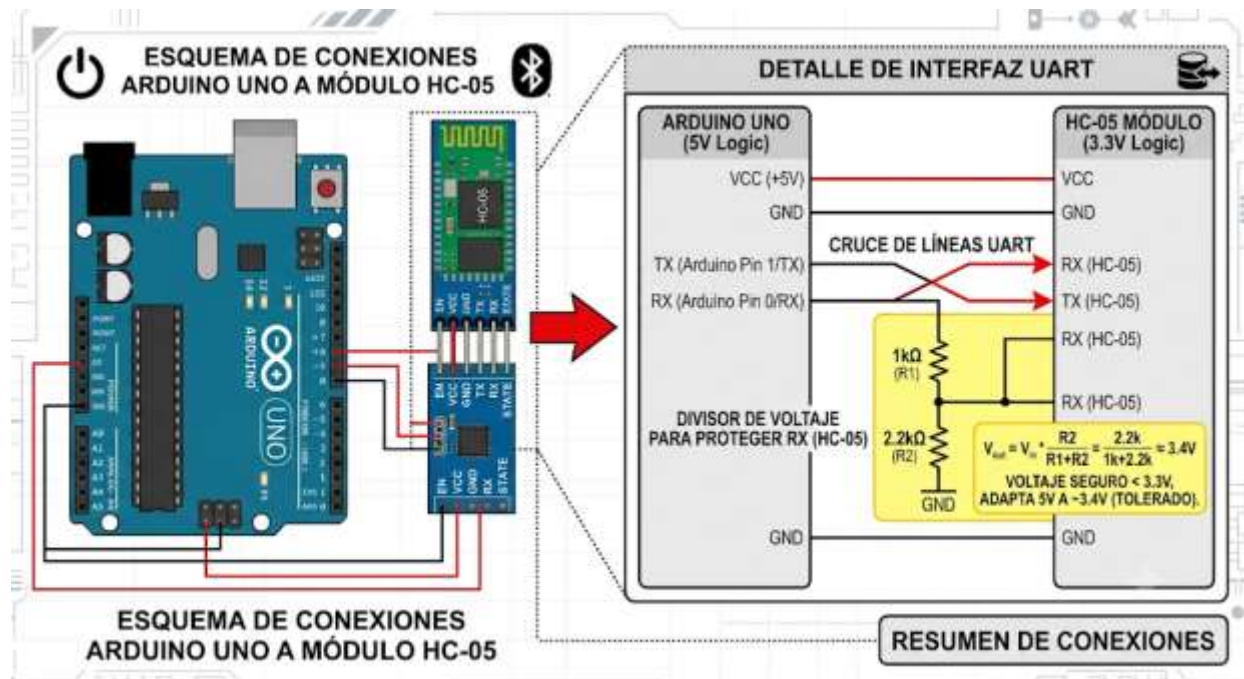

```
}  
// Pausa la ejecución del programa durante 500 milisegundos para estabilizar las  
lecturas delay(500);  
}
```

4.3. MÓDULOS DE CONECTIVIDAD INALÁMBRICA (BLUETOOTH HC-05 O ESP32/WIFI)

Una vez que controlamos el hardware de forma local, el siguiente paso como Técnicos es dotarlo de capacidad de comunicación inalámbrica. En la práctica real existen dos tecnologías principales para este propósito en entornos educativos y de pequeña industria:

1. **Módulos Bluetooth (como el HC-05 / HC-06):** Operan bajo la frecuencia de 2.4 GHz y actúan como un puente serie inalámbrico. Utilizan el protocolo UART (Transmisión-Recepción Asíncrona Universal) conectándose a los pines TX (Transmisión) y RX (Recepción) del microcontrolador. Son ideales para conexiones de corto alcance (hasta 10 metros) dentro de los talleres del CEA, permitiendo que un teléfono se enlace directamente al módulo sin necesidad de un enrutador intermedio.
2. **Módulos ESP32 (Ecosistema con WiFi nativo):** Son microcontroladores avanzados de 32 bits que integran conectividad WiFi y Bluetooth en el mismo chip. Permiten conectar el hardware directamente a la red inalámbrica local instalada en el Barrio Residencial Norte o enviar datos a servidores en la nube mediante protocolos de Internet.

La comunicación serie se basa en el envío secuencial de bits a través de un canal físico o inalámbrico. Cuando el módulo Bluetooth de un negocio en el Barrio Central Fátima recibe un carácter desde el teléfono celular del dueño, el microcontrolador almacena ese byte en un espacio de memoria temporal denominado *Buffer*. El software desarrollado por el Técnico debe inspeccionar constantemente este buffer para procesar e interpretar los comandos recibidos.



El siguiente código demuestra cómo configurar el microcontrolador para escuchar activamente comandos de control inalámbricos que provienen de la aplicación móvil:

C++

```
// Variable de tipo caracter para almacenar el byte recibido por el canal
// inalámbricochar comandoRecibido;
const int pinLuzPatio = 12;
// Pin de salida para el sistema de iluminación exteriorvoid setup() {
// Inicializa el puerto serie principal a 9600 baudios (velocidad de transmisión
// inalámbrica) Serial.begin(9600);
// Establece el pin de la luz exterior como salida para operar el relé de
// potencia pinMode(pinLuzPatio, OUTPUT);
// Asegura que la luz inicie apagada al arrancar el circuito
digitalWrite(pinLuzPatio, LOW);
}
void loop() {
// Verifica si existen bytes disponibles para ser leídos en el buffer del puerto
// serie if (Serial.available() > 0) {
// Lee el byte más antiguo almacenado en el buffer de recepción
comandoRecibido = Serial.read();
// Evalúa el caracter recibido para ejecutar la acción correspondiente en el CEA
if (comandoRecibido == '
A') {
// Si el software móvil envía la Letra '
A', se energiza el pin para encender las luces digitalWrite(pinLuzPatio,
HIGH);
```



```
Serial.println("
Comando Recibido: Luces exteriores ENCENDIDAS.");
}
else if (comandoRecibido == '
B') {
// Si el software móvil envía la letra '
B', se corta el voltaje para apagar las luces      digitalWrite(pinLuzPatio,
LOW);
Serial.println("
Comando Recibido: Luces exteriores APAGADAS.");
}
}
}
```

4.4. DESARROLLO DE INTERFAZ MÓVIL PARA ENVÍO DE COMANDOS DE CONTROL

La programación móvil orientada al IoT demanda interfaces intuitivas y un manejo óptimo de los hilos de comunicación de datos. En el ámbito del desarrollo para sistemas Android, la arquitectura se fundamenta en componentes visuales y lógicos que interactúan de manera asíncrona para no congelar la pantalla del usuario mientras se busca o transmite información por el adaptador Bluetooth o la conexión de red.

Una interfaz bien diseñada para los usuarios de Pailón debe proporcionar controles claros (botones, selectores) y un indicador visual del estado actual de la conexión. Técnicamente, el flujo operativo dentro de la aplicación móvil requiere:

1. **Solicitud de Permisos:** El sistema operativo móvil exige que el usuario apruebe el uso del hardware de comunicación (Bluetooth o WiFi) y el acceso a la ubicación detallada del dispositivo.
2. **Descubrimiento y Selección:** Desplegar una lista o menú desplegable con las direcciones MAC únicas de los dispositivos emparejados en las cercanías del Barrio Saó.
3. **Apertura del Socket de Comunicación:** Establecer un canal lógico seguro de flujo de entrada y salida (*Input/Output Stream*) a través del cual se inyectarán los bytes de comando.

El fragmento de código estructurado en lenguaje Kotlin detalla la lógica de control del lado de la aplicación móvil para conectarse al hardware del CEA y realizar el envío seguro de caracteres de control:



Kotlin

```

package com.cea.iotcontrolimport android.bluetooth.BluetoothAdapterimport
android.bluetooth.BluetoothDeviceimport android.bluetooth.BluetoothSocketimport
android.os.Bundleimport android.widget.Buttonimport android.widget.Toastimport
androidx.appcompat.app.AppCompatActivityimport java.io.IOExceptionimport
java.io.OutputStreamimport java.util.UUIDclass MainActivity :
AppCompatActivity() {
    // Adaptador nativo del sistema para gestionar las funciones de radio Bluetooth
    del teléfono     private var adaptadorBluetooth: BluetoothAdapter? = null    //
    Socket Lógico que mantendrá la sesión de datos activa con el microcontrolador
    private var socketConexion: BluetoothSocket? = null    // Flujo de salida para
    empujar los bytes de comando hacia el hardware de Pailón     private var
    flujoSalida: OutputStream? = null    // Identificador Único Universal estándar
    (UUID) para el perfil de puerto serie (SPP)     private val uuidSpp: UUID =
    UUID.fromString("00001101-0000-1000-8000-00805F9B34FB")    // Dirección física
    MAC de prueba del módulo instalado en el CEA Herman Ortiz Camargo     private val
    direccionMacModulo = "00:21:13:01:48:84"    override fun
    onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)                setContentView(layout.activity_main)
        // Instancia los botones definidos en el archivo de diseño de interfaz de
        usuario         val botonConectar = findViewById<Button>(R.id.btnConectar)
        val botonEncender = findViewById<Button>(R.id.btnEncender)         val
        botonApagar = findViewById<Button>(R.id.btnApagar)         // Obtiene La
        referencia del adaptador por defecto del dispositivo inteligente
        adaptadorBluetooth = BluetoothAdapter.getDefaultAdapter()         // Asigna la
        acción de clic para iniciar el enlace inalámbrico hacia el Barrio Saó
        botonConectar.setOnClickListener {
            conectarAlMicrocontrolador()        }
        // Asigna la acción para transmitir el carácter de encendido ('
        A')         botonEncender.setOnClickListener {
            enviarComandoInalambrico('
        A')        }
        // Asigna la acción para transmitir el carácter de apagado ('
        B')         botonApagar.setOnClickListener {
            enviarComandoInalambrico('
        B')        }
    }
    private fun conectarAlMicrocontrolador() {
        // Obtiene la representación del dispositivo remoto usando su dirección de
        hardware MAC         val dispositivoRemoto: BluetoothDevice? =
        adaptadorBluetooth?.getRemoteDevice(direccionMacModulo)         try {
        // Intenta crear un canal de comunicación de datos seguro utilizando el estándar
        SPP         socketConexion =
        dispositivoRemoto?.createRfcommSocketToServiceRecord(uuidSpp)         //
        Detiene cualquier proceso de búsqueda activa de dispositivos para liberar
        recursos de hardware         adaptadorBluetooth?.cancelDiscovery()
        // Inicia formalmente el enlace de conexión. Bloquea el hilo hasta tener éxito o
    
```



```

fallar          socketConexion?.connect()          // Extrae el flujo de
datos de salida para permitir la escritura posterior de comandos
flujoSalida = socketConexion?.outputStream          Toast.makeText(this, "
Conexión Exitosa con el módulo del CEA", Toast.LENGTH_SHORT).show()      }
catch (e: IOException) {
// Manejo de excepciones en caso de fallos de energía o distancia excesiva en el
entorno          Toast.makeText(this, "
Error Crítico: No se pudo establecer el enlace físico",
Toast.LENGTH_SHORT).show()          try {
socketConexion?.close() }
catch (closeException: IOException) {
}
}
}
private fun enviarComandoInalambrico(caracterComando: Char) {
// Verifica que el canal de salida esté debidamente instanciado y activo
if (flujoSalida != null) {
try {
// Convierte el carácter a su equivalente en formato de bytes y lo empuja al
canal          flujoSalida?.write(caracterComando.toInt())      }
catch (e: IOException) {
Toast.makeText(this, "
Error al transmitir comando a la instalación", Toast.LENGTH_SHORT).show()
}
}
}
}
}

```

4.5. RECEPCIÓN DE DATOS DE SENSORES DESDE EL HARDWARE A LA APLICACIÓN MÓVIL

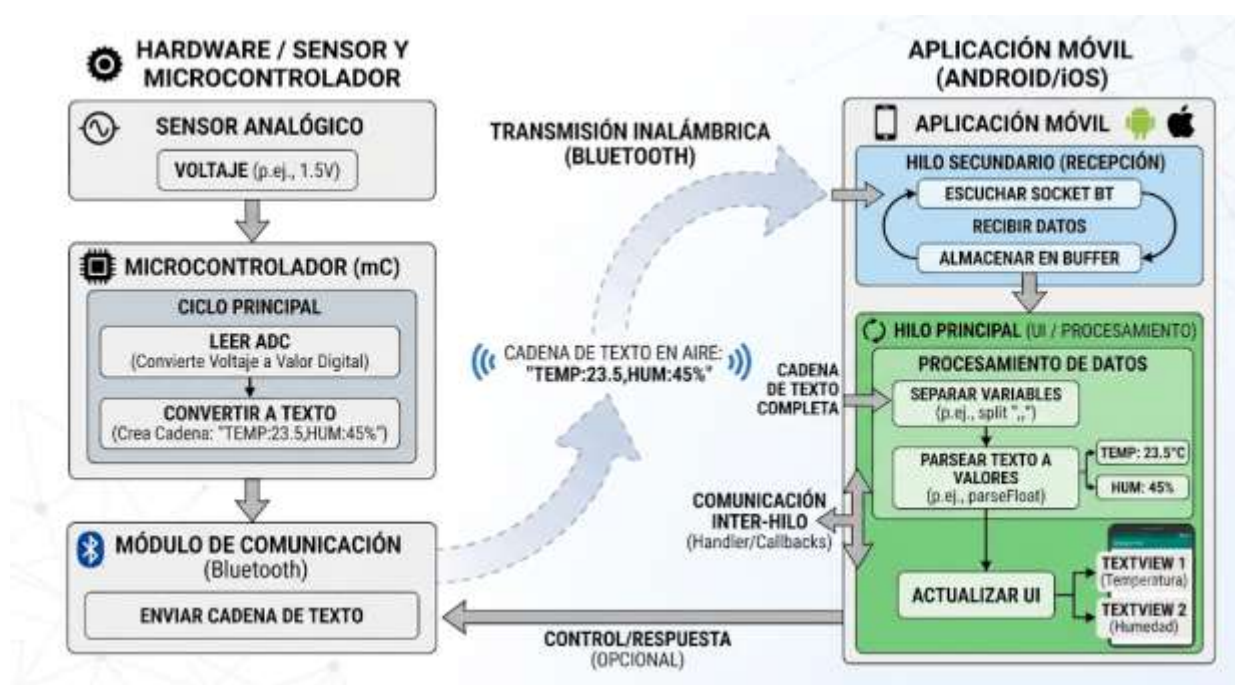
El verdadero potencial del IoT se despliega cuando la aplicación móvil no solo actúa como un control remoto, sino como un centro de telemetría y monitoreo capaz de recibir, procesar y representar gráficamente los datos recopilados por sensores remotos. En nuestro escenario de Pailón, esto implica leer los niveles de temperatura ambiente de los laboratorios del Barrio Saó o el estado de refrigeración en la estación del Barrio 24 de Septiembre.

Para lograr esto, el microcontrolador debe leer de forma periódica el valor analógico o digital de un sensor, empaquetar dicha información dentro de una cadena de texto (*String*) estructurada y transmitirla por el puerto serie inalámbrico. Es una práctica fundamental de la ingeniería de software definir un "delimitador" o carácter de fin de línea (como el salto de línea \n). Esto permite



que la aplicación móvil sepa con total certeza en qué momento termina un paquete de datos y comienza el siguiente.

Del lado del teléfono inteligente, el proceso de lectura no puede realizarse de forma secuencial directa en la interfaz gráfica, ya que la espera de datos bloquearía la aplicación. Se requiere implementar un **Hilo Secundario de Ejecución (Thread)** que corra de manera permanente en segundo plano. Este hilo escucha de forma asíncrona los bytes entrantes desde el flujo de entrada (*InputStream*), concatena los caracteres y, al detectar el delimitador de fin de línea, envía el mensaje completo al hilo principal para actualizar los elementos visuales de la interfaz de usuario.



A continuación, se detalla el código C++ adaptado para el hardware del microcontrolador encargado de leer un sensor y realizar el envío continuo de métricas hacia el dispositivo móvil:

C++

```
const int pinSensorTemperatura = A0;
// Pin analógico seleccionado para leer el sensor de temperatura float
temperaturaCalculada = 0.0;
// Variable decimal para almacenar el valor físico real int lecturaCruda = 0;
// Variable para el valor digital directo del convertidor (0 a 1023)
void setup() {
// Inicializa el puerto serie a 9600 baudios para garantizar compatibilidad con
hilos móviles Serial.begin(9600);
}
```



```
void loop() {  
  // Lee el voltaje del pin analógico A0 (Retorna un número entero entre 0 y 1023  
  // debido al ADC de 10 bits)  LecturaCruda = analogRead(pinSensorTemperatura);  
  // Realiza la conversión matemática para traducir el valor digital a grados  
  // Celsius reales  // (Fórmula específica basada en un sensor estándar lineal como  
  // el LM35, adaptado a los niveles de tensión)  temperaturaCalculada =  
  (LecturaCruda * 5.0 * 100.0) / 1024.0;  
  // Transmite el valor calculado al puerto serie en formato de texto plano  
  Serial.print(temperaturaCalculada);  
  // Envía de forma inmediata el carácter delimitador crítico de fin de mensaje  
  // (\n) mediante println  // Esto le indica al hilo secundario de Android en el  
  // Barrio Saó que el paquete numérico ha terminado  Serial.println();  
  // Configura un intervalo de muestreo de 2000 milisegundos para evitar la  
  // saturación del canal inalámbrico  delay(2000);  
}
```

4.6. INTEGRACIÓN DE INTELIGENCIA ARTIFICIAL PERIMETRAL (EDGE AI) EN MICROCONTROLADORES IOT

La evolución tecnológica actual exige que los sistemas informáticos embebidos dejen de ser meros recolectores de datos pasivos y adquieran capacidades de procesamiento avanzado local. La Inteligencia Artificial Perimetral, conocida técnicamente como *Edge AI* o *TinyML* (Machine Learning en Microcontroladores), consiste en la ejecución de modelos matemáticos optimizados de aprendizaje automático directamente dentro del silicio de dispositivos de baja potencia, como los microcontroladores que manejas como Técnico.

Anteriormente, para predecir si una bomba de agua en los huertos comunales de los Jardines de Pailón o un motor de refrigeración ferroviario en el Barrio 24 de Septiembre iba a fallar, era obligatorio transmitir millones de muestras de vibración y temperatura a un servidor en la nube. Esto conllevaba altos costos de ancho de banda y latencias inaceptables. Con TinyML, un modelo de red neuronal artificial o un árbol de decisión previamente entrenado en una computadora de escritorio se comprime, cuantifica (reduciendo el formato numérico de flotantes de 32 bits a enteros de 8 bits) y se exporta como un arreglo de constantes en lenguaje C++ para ser flasheado directamente en el microcontrolador.

Esto permite que el hardware realice análisis predictivos en tiempo real a nivel local. Por ejemplo, al leer el perfil de corriente eléctrica de los laboratorios del CEA Herman Ortiz Camargo, el chip puede deducir de forma autónoma si hay un comportamiento anómalo (como un cortocircuito



sutil o un equipo funcionando de forma ineficiente) y disparar alertas directas a la aplicación móvil del Técnico sin requerir conexión externa de red o servidores en internet.

4.7. PROTOCOLOS DE COMUNICACIÓN EFICIENTES (MQTT) PARA MONITOREO EN REDES DE SENSORES

Cuando la escala de un proyecto informático se expande y dejamos de monitorizar un único dispositivo para integrar múltiples nodos distribuidos en los barrios de Pailón (por ejemplo, estaciones de temperatura en San Antonio, monitores de humedad en Las Misiones y sensores de presencia en el Barrio María Belén), la comunicación serie directa y el protocolo HTTP convencional se vuelven ineficientes debido a su alta sobrecarga de datos (*overhead*). Es aquí donde entra en juego el protocolo industrial **MQTT** (Message Queuing Telemetry Transport).

MQTT es un protocolo de red de mensajería liviano basado en el patrón de arquitectura **Publicación/Suscripción** (Pub/Sub), diseñado específicamente para ejecutarse sobre la pila TCP/IP en entornos con recursos de hardware limitados o conexiones de red inestables. En este modelo no existen las conexiones directas cliente-servidor tradicionales. En su lugar, se dispone de un nodo central llamado **Broker** (que puede correr en un servidor local dentro del nuevo módulo equipado en el Barrio Saó).

Los microcontroladores actúan como clientes que "publican" mensajes con datos de sensores en canales de texto específicos denominados **Tópicos** (ejemplo: pailon/cea/temperatura). Por otro lado, la aplicación móvil desarrollada por el Técnico se "suscribe" a ese mismo tópico. En el instante exacto en que un nodo publica una nueva lectura, el Broker la distribuye de forma inmediata a todos los teléfonos suscritos. Este protocolo optimiza drásticamente el consumo de batería del celular y reduce el tamaño de los paquetes de datos a unos pocos bytes de cabecera.

4.8. SEGURIDAD, ENCRIPCIÓN Y MITIGACIÓN DE VULNERABILIDADES EN CONEXIONES MÓVILES IOT

Un aspecto crítico y frecuentemente ignorado en el despliegue de soluciones de microcontroladores y programación móvil es la seguridad de la información. Los dispositivos IoT son el blanco principal de ciberataques debido a que carecen de las capas defensivas tradicionales de los sistemas operativos robustos. En el entorno de Pailón, una vulnerabilidad en



el sistema de control inalámbrico del CEA podría permitir que un intruso con conocimientos básicos intercepte las señales de radio y abra las cerraduras electrónicas o apague deliberadamente los sistemas informáticos de protección del laboratorio.

Para mitigar estos riesgos de seguridad, el Técnico en Sistemas Informáticos debe implementar estrategias de protección en tres niveles específicos:

1. **Autenticación Robusta:** Requerir mecanismos de desafío-respuesta al abrir el socket de conexión. El microcontrolador no debe ejecutar ningún comando proveniente del software móvil si este no incluye un token criptográfico dinámico o una clave hash precompartida que verifique la identidad del operador de seguridad.
2. **Encriptación de Datos:** Los canales Bluetooth clásicos sin configuraciones adicionales transmiten la información en formato de texto plano legible por cualquier antena interceptora en las cercanías. Se deben emplear algoritmos de cifrado simétrico ligero como AES-128 o ChaCha20 directamente en el microcontrolador y en la app móvil. De esta forma, el comando para activar las luces no viaja como una simple letra 'A', sino como una cadena cifrada no legible por terceros.
3. **Ofuscación y Validación de Entradas:** Sanitizar y validar sintácticamente cada byte que ingresa al buffer serie del microcontrolador. Esto evita ataques de desbordamiento de memoria (*Buffer Overflow*) diseñados para congelar el chip o forzarlo a ejecutar código arbitrario perjudicial.

CIERRE TEÓRICO OBLIGATORIO

❑ Mitos vs. Realidades

Mito	Realidad
Para implementar soluciones de Internet de las Cosas (IoT) en empresas o instituciones es obligatorio contar con una conexión a Internet de banda ancha fija las 24 horas del día.	Falso. Muchas infraestructuras IoT operan eficientemente de forma perimetral local utilizando conexiones directas inalámbricas de corto alcance como Bluetooth, redes de sensores locales o servidores MQTT locales instalados directamente en el área física de trabajo sin salida a la web.



Los microcontroladores de hardware libre carecen de la robustez necesaria y solo sirven para proyectos educativos de nivel básico o maquetas escolares.

Totalmente falso. Los microchips empleados en estos ecosistemas tienen la misma arquitectura que los componentes que controlan sistemas de inyección automotriz y automatización industrial, demostrando alta resiliencia térmica, eléctrica y operativa si se protegen con circuitos de aislamiento adecuados.

□ Glosario de Términos

1. **Microcontrolador:** Circuito integrado programable que concentra una CPU, memoria y periféricos de entrada/salida en un único chip para ejecutar una tarea dedicada de software de manera cíclica e indefinida.
2. **Baudios (Baud rate):** Unidad de medida que representa la velocidad de transmisión de datos en un canal de comunicación en serie, indicando el número de cambios de estado o símbolos transmitidos por segundo (ej. 9600 baudios).
3. **Socket:** Punto final lógico de un enlace de comunicación bidireccional entre dos programas que se ejecutan a través de la red o interfaces inalámbricas, permitiendo la lectura y escritura de flujos de datos continuos.
4. **GPIO (General Purpose Input/Output):** Pines de propósito general en un circuito integrado que pueden ser configurados por el programador mediante software para actuar tanto como entradas de lectura eléctrica como salidas de control de voltaje.
5. **Asincronismo:** Método de ejecución de procesos computacionales donde las operaciones se realizan de forma independiente al flujo del hilo principal del sistema, evitando que tareas de larga duración (como buscar y conectar un canal Bluetooth) bloqueen la interfaz gráfica de usuario.

□ Ponte a Prueba

1. Cuando configuramos un pin digital del microcontrolador utilizando la instrucción específica `pinMode(pin, INPUT_PULLUP)`, ¿cuál es el estado eléctrico por defecto que leerá el pin cuando el circuito del sensor esté completamente abierto y desconectado de tierra?



1. A) Leerá un estado inestable y oscilante debido al ruido electromagnético ambiental de la zona.
 2. B) Leerá firmemente un estado BAJO (LOW o 0 Voltios).
 3. C) Leerá de manera segura y estable un estado ALTO (HIGH o voltaje de alimentación).
-
2. ¿Cuál es el rol técnico fundamental que cumple el carácter especial de salto de línea \n cuando el microcontrolador envía lecturas numéricas de sensores a una aplicación móvil Android?
-
1. A) Aumentar la velocidad física de transmisión por radio del módulo Bluetooth.
 2. B) Actuar como un delimitador o marcador de fin de paquete, indicando al hilo secundario de la app móvil que la lectura actual se completó y ya puede procesarse en pantalla.
 3. C) Cifrar y proteger la información contra ataques de interceptación inalámbrica en los alrededores.
-
3. En el desarrollo de software móvil para IoT, ¿por qué es una regla crítica de arquitectura de sistemas informáticos ejecutar la búsqueda de hardware y la lectura del canal de datos en un Hilo Secundario (*Thread*) en lugar del Hilo Principal (*UI Thread*)?
-
1. A) Porque el hilo principal del teléfono inteligente carece por completo de los permisos de hardware para conectarse a las antenas internas de Bluetooth o WiFi.
 2. B) Para evitar que las esperas prolongadas de datos entrantes congelen la interfaz visual de la aplicación, garantizando una experiencia de usuario fluida y libre de bloqueos.
 3. C) Para reducir obligatoriamente el consumo de energía eléctrica de la batería del dispositivo móvil del Técnico.

(Respuestas correctas: 1: C, 2: B, 3: B)



VALORACIÓN



REFLEXIÓN SOCIO-TECNOLÓGICA EN LA REGIÓN DE CHIQUITOS

La implementación acelerada de sistemas de automatización, robótica e Internet de las Cosas (IoT) plantea profundas interrogantes que van mucho más allá de la simple optimización técnica, y como Técnicos en Sistemas Informáticos formados en el CEA Herman Ortiz Camargo debes ser plenamente conscientes de ello. El despliegue de estas tecnologías en una población en crecimiento como Pailón genera un impacto directo en tres ejes fundamentales:

En primer lugar, la **sostenibilidad medioambiental y la gestión de residuos electrónicos** (e-waste). El uso extensivo de sensores, microcontroladores y baterías de litio de bajo costo para automatizar los campos agrícolas o los comercios del Barrio Central Fátima conlleva la responsabilidad de planificar el ciclo de vida de dichos componentes. El desecho irresponsable de estos dispositivos en los alrededores del Barrio Saó o su acumulación en vertederos comunes libera metales pesados que pueden contaminar las napas freáticas de agua dulce de la región. Como profesionales de sistemas, tu deber ético incluye diseñar carcasas duraderas, utilizar componentes reutilizables y promover planes de reciclaje tecnológico local.

En segundo lugar, se debe evaluar el **impacto social en el empleo local y el desarrollo comunitario**. Si bien la automatización de los centros de almacenamiento cercanos a la estación de trenes optimiza la productividad de las empresas, existe el riesgo latente de desplazar mano de obra local no calificada dedicada a tareas de monitoreo manual. Por ende, la tecnología no debe concebirse como un reemplazo del ser humano, sino como una herramienta de amplificación de capacidades. El software que desarrolles debe capacitar a los trabajadores locales de Pailón, transformando sus roles de vigilantes manuales a operadores de plataformas tecnológicas avanzadas.

Finalmente, la **privacidad y seguridad de los datos civiles**. Al digitalizar el monitoreo de un espacio educativo o comercial, estamos recolectando patrones de comportamiento de las personas: horarios de ingreso, niveles de consumo y flujos de tránsito en los barrios. La desatención en la seguridad del software embebido que permita fugas de información



inalámbrica vulnera directamente el derecho a la privacidad de la comunidad. La ética profesional exige que todo sistema implementado garantice que los datos recopilados sean tratados con confidencialidad, transparencia y bajo el consentimiento explícito de los usuarios.



PRODUCCIÓN



LABORATORIO PRÁCTICO: SISTEMA IOT DE CONTROL PERIMETRAL Y ALERTA TÉRMICA PARA EL CEA HERMAN ORTIZ CAMARGO

Objetivo del Laboratorio: Diseñar, conectar y programar de forma integrada un prototipo de hardware-software que simule el control del sistema de ventilación del laboratorio de cómputo en el Barrio Saó. El sistema usará un microcontrolador para sensorar datos térmicos, reaccionar a comandos remotos vía serie inalámbrica enviados desde una app móvil en Android y reportar de vuelta el estado de seguridad perimetral de la estructura en tiempo real.

Equipamiento Requerido (Simulado o Físico en Computadora)

1. 1 Placa Microcontroladora compatible con entorno Arduino (Uno, Nano o ESP32).
2. 1 Módulo de comunicación inalámbrica Bluetooth (HC-05) o Cable de interfaz serie USB.
3. 1 Sensor de Temperatura de precisión lineal (LM35 o equivalente).
4. 1 Interruptor de contacto magnético (o un pulsador estándar para simular la puerta).
5. 1 Diodo LED de color rojo (Alerta) y un módulo de Relé acoplado a un ventilador pequeño de 5V.
6. Entorno de Desarrollo: Arduino IDE instalado en la computadora y Android Studio para la lógica móvil.



Guía de Ejecución Paso a Paso

Paso 1: Ensamblado del Hardware del Sistema de Control

Realiza el montaje del circuito respetando la distribución de energía. Conecta el pin de datos del sensor de temperatura a la entrada analógica A0 del microcontrolador. Conecta el interruptor magnético (simulador de puerta) al pin digital 2. El diodo LED de alerta perimetral y el pin de activación del relé del ventilador deben conectarse a las salidas digitales 12 y 13 respectivamente. Cruza las líneas del módulo de comunicación inalámbrica: la línea de transmisión del módulo irá conectada a la recepción del controlador (RX) y viceversa, asegurando compartir una línea común de tierra (GND).

Paso 2: Desarrollo e Inyección de Firmware en el Microcontrolador

Abre el editor Arduino IDE, transcribe el siguiente código fuente integral detallado a continuación, compíllalo para verificar la ausencia de errores de sintaxis y súbelo a la memoria flash de tu placa microcontroladora a través del puerto USB:

C++

```
// Definición técnica de pines para periféricos del CEA
const int pinSensorTemp = A0;
const int pinPuerta = 2;
const int pinAlertaLed = 12;
const int pinReleVentilador = 13;
// Variables de estado interno del sistema
float temperaturaCelsius = 0.0;
bool alarmaActivada = false;
char comandoMovil = ' ';
void setup() {
    // Inicializa la velocidad del puerto serie para sincronización con el hilo móvil
    Serial.begin(9600);
    // Configuración de modos de pin
    pinMode(pinPuerta, INPUT_PULLUP);
    pinMode(pinAlertaLed, OUTPUT);
    pinMode(pinReleVentilador, OUTPUT);
    // Estado inicial seguro del hardware en el Barrio Saó
    digitalWrite(pinAlertaLed, LOW);
    digitalWrite(pinReleVentilador, LOW);
}
void loop() {
    // 1. Monitoreo Perimetral (Lectura de entrada digital)
    int estadoPuerta = digitalRead(pinPuerta);
    // Evaluamos si el interruptor magnético detecta la puerta abierta (HIGH)
    if
```




```

(estadoPuerta == HIGH) {
  alarmaActivada = true;
  digitalWrite(pinAlertaLed, HIGH);
  // Enciende la luz roja de alerta de intrusión }
else {
  alarmaActivada = false;
  digitalWrite(pinAlertaLed, LOW);
}
// 2. Telemetría Térmica (Lectura analógica y conversión matemática) int
valorCrudo = analogRead(pinSensorTemp);
temperaturaCelsius = (valorCrudo * 5.0 * 100.0) / 1024.0;
// 3. Escucha Activa de Comandos Inalámbricos desde la aplicación móvil if
(Serial.available() > 0) {
  comandoMovil = Serial.read();
  // Comando para encender manualmente el extractor de aire desde el celular if
  (comandoMovil == '
X') {
    digitalWrite(pinReleVentilador, HIGH);
  }
  // Comando para apagar manualmente el extractor de aire desde el celular else
  if (comandoMovil == '
Y') {
    digitalWrite(pinReleVentilador, LOW);
  }
}
// 4. Empaquetamiento y Transmisión de Datos seriales hacia Android en Pailón
// El paquete estructurado viaja en formato plano: "
Temperatura,EstadoAlarma" Serial.print(temperaturaCelsius);
Serial.print(",");
Serial.print(alarmaActivada ? "1" : "0");
// Inyección obligatoria del delimitador de fin de línea para el hilo secundario
Serial.println();
// Retardo de estabilidad operativa del lazo de control delay(1000);
}

```

Paso 3: Diseño de la Interfaz Gráfica de la Aplicación Móvil

En tu entorno de desarrollo Android Studio, edita el archivo de interfaz visual en formato XML para crear una pantalla clara para el Técnico del CEA. Añade etiquetas de texto grandes para visualizar el valor actual de la temperatura medida en Pailón, un indicador visual de color para el estado de la puerta del laboratorio informático, un selector de dispositivos de comunicación de radio y dos botones de control industrial etiquetados como "Activar Ventilación Local" y "Desactivar Ventilación Local".



Paso 4: Programación de la Lógica del Hilo de Recepción Asíncrona

Implementa en el código lógico de tu aplicación móvil Android (MainActivity) la estructura de manejo de datos de red e hilos secundarios. El siguiente fragmento describe conceptualmente el bloque del hilo de recepción que debes acoplar a la lógica de Kotlin para procesar la telemetría enviada por el microcontrolador desde el Barrio Saó:

Kotlin

```
// Hilo secundario dedicado de forma exclusiva a escuchar el buffer de datos
inalámbricosfun iniciarHiloEscuchaDatos(socket: BluetoothSocket) {
    val flujoEntrada = socket.inputStream    val bufferLectura = ByteArray(1024)
    var bytesDisponibles: Int    var datosAcumuladosTexto = ""    // Bucle infinito
    que se ejecuta en segundo plano sin interferir con la pantalla táctil    Thread
    {
        while (true) {
            try {
                // Lee los bytes que van ingresando al buffer físico de radio del teléfono
                celular        bytesDisponibles = flujoEntrada.read(bufferLectura)
                if (bytesDisponibles > 0) {
                    // Convierte el fragmento de bytes recibidos a texto plano legible
                    val fragmentoTexto = String(bufferLectura, 0, bytesDisponibles)
                    datosAcumuladosTexto += fragmentoTexto    // Verifica si en el
                    texto acumulado ya se encuentra el delimitador crítico (\n)
                    if (datosAcumuladosTexto.contains("\n")) {
                        // Extrae la línea de datos completa quitando los espacios sobrantes
                        val lineaCompletaDatagrama = datosAcumuladosTexto.substringBefore("\n").trim()
                        // Limpia el acumulador para prepararlo para recibir el siguiente segundo de
                        telemetría        datosAcumuladosTexto =
                        datosAcumuladosTexto.substringAfter("\n")    // Realiza el
                        procesamiento de datos (*Parsing*) separando por la coma estructural
                        val variablesSeparadas = lineaCompletaDatagrama.split(",")
                        if (variablesSeparadas.size == 2) {
                            val valorTemperaturaRecibida = variablesSeparadas[0]
                            val estadoAlarmaRecibida = variablesSeparadas[1]    //
                            Ordena al hilo principal de la interfaz actualizar los textos visuales del
                            usuario        runOnUiThread {
                                actualizarInterfazUsuario(valorTemperaturaRecibida, estadoAlarmaRecibida)
                            }
                        }
                    }
                }
            } catch (e: IOException) {
                // Control preventivo de interrupciones físicas por distancia en los pasillos de
                Pailón        break    }
            }
        }
    }
}
```



```
}  
}  
.start()  
fun actualizarInterfazUsuario(temp: String, alarma: String) {  
    // Código para enlazar los datos de texto recibidos a las etiquetas visuales de  
    // la interfaz de usuario de Android // Si alarma == "1", la etiqueta de  
    // seguridad se pinta de color rojo alertando intrusión en el módulo educativo.}
```

Paso 5: Pruebas de Integración y Validación del Sistema en Pailón

Energiza el circuito de hardware. Desde el menú de configuraciones de tu teléfono inteligente, vincula el adaptador inalámbrico con el módulo del microcontrolador. Inicia la aplicación móvil desarrollada y presiona el botón de enlace para abrir el socket de comunicación. Observa cómo cada segundo se actualiza automáticamente el valor de temperatura en la pantalla de tu teléfono.

Prueba desconectar el interruptor magnético de la puerta en el Barrio Saó: la aplicación móvil debe cambiar inmediatamente su estado visual a un formato de alerta roja parpadeante. Presiona los botones de control de ventilación y confirma que el relé físico responde de forma instantánea activando el extractor de aire del laboratorio, consolidando así el correcto funcionamiento del ecosistema IoT perimetral.

RECURSOS ADICIONALES

1. **Documentación Oficial de Referencia de Arduino:** El portal técnico principal para profundizar de manera exhaustiva en la sintaxis del lenguaje, funciones avanzadas del manejo de registros del microcontrolador y el uso correcto de librerías de hardware. Disponible en la plataforma oficial del ecosistema web de Arduino.
2. **Repositorio y Guías de Conectividad Inalámbrica en Android Developers:** El sitio centralizado de Google para ingenieros de software móvil donde se explican los estándares técnicos de control de Bluetooth clásico, Bluetooth Low Energy (BLE) y la gestión de permisos asíncronos en las últimas versiones del sistema operativo móvil.
3. **Manuales de Sensores e Interfaces Electrónicas de SparkFun / Adafruit Learning System:** Repositorios de acceso libre con guías visuales excelentes, esquemáticos eléctricos detallados, especificaciones técnicas de circuitos integrados y metodologías de



aislamiento de potencia ideales para el diseño de placas de circuito en entornos industriales y civiles.